



UNIVERSITÀ
DEGLI STUDI
DI TORINO

Laboratorio Algoritmi e Strutture Dati
2024/2025

Luca Gado, Mehratab Istifanos

Contenuti

Esercizio 1: Merge Sort e Quick Sort

Esercizio 2: Edit Distance

Esercizio 3: PriorityQueue

Esercizio 4: Grafi sparsi e foreste ricoprenti minime

Esercizio 1: Merge Sort e Quick Sort

Ho fatto una implementazione di una libreria di ordinamento con MergeSort e QuickSort e un programma (**main_ex1**) che ordina un file CSV di record nei tre campi:

- field1 (stringa)
- field2 (intero)
- field3 (double)

È stato realizzato un file di test con Unity (**test_ex1**), che copre:

- ordinamento corretto su array di int, double e stringhe,
- casi limite (array vuoto, singolo elemento, duplicati, già ordinato, inverso),
- policy sui valori speciali (NULL stringhe < qualsiasi stringa; NaN double in fondo),
- verifica di stabilità: MergeSort mantiene l'ordine relativo degli elementi con chiavi uguali, QuickSort no.

Risultato: 11 test eseguiti, 0 failure.

Dataset: file records.csv (in questo test è stato utilizzato un sottoinsieme, non l'intero dataset da 20M per limiti di tempo).

Sistema: Windows 11, compilazione con GCC (MinGW), esecuzione via PowerShell.

Per ogni combinazione (campo × algoritmo) sono stati eseguiti 3 run, misurando il tempo totale (I/O + sort) con cronometro Stopwatch.

Calcolati media e deviazione standard.

In caso di esecuzione >600s, sarebbe stato annotato fail (non si è verificato in questi test).

Caso	Field	Algo	Run1 (s)	Run2 (s)	Run3 (s)	Media (s)	Dev.st. (s)
f1_merg e	1	1	0,068	0,017	0,016	0,034	0,030
f1_quick	1	2	0,017	0,037	0,016	0,023	0,012
f2_merg e	2	1	0,010	0,013	0,013	0,012	0,002
f2_quick	2	2	0,012	0,012	0,010	0,011	0,001
f3_merg e	3	1	0,011	0,013	0,036	0,020	0,014
f3_quick	3	2	0,022	0,021	0,028	0,024	0,004

- field1 (stringhe): QuickSort leggermente più rapido (0,023s vs 0,034s) ma con deviazione standard più bassa per MergeSort → Merge è più stabile, Quick è più veloce in questo test ma potenzialmente vulnerabile a pivot sfortunati.
- field2 (interi): MergeSort e QuickSort molto simili (0,012s vs 0,011s), differenze trascurabili.
- field3 (double): MergeSort più variabile (dev.st. 0,014), QuickSort più costante e leggermente più veloce (0,024s vs 0,020s in media).

Nota: i tempi sono in secondi, molto ridotti perché il dataset usato era un sottoinsieme del file completo (non i 20M record). Su input più grandi, le differenze tra algoritmi si amplificherebbero, in particolare la stabilità di MergeSort rispetto ai peggiori casi di QuickSort.

- MergeSort: complessità $O(n \log n)$, prestazioni regolari anche su distribuzioni sfavorevoli.
- QuickSort: caso medio $O(n \log n)$ ma peggioramento a $O(n^2)$ se il pivot produce partizioni sbilanciate. Nei test si è visto leggermente più veloce, ma con dataset grandi può fallire o degradare.

I run sperimentali (su dataset ridotto) mostrano tempi compatibili con la teoria MergeSort e QuickSort.

Su dataset molto grandi, ci si aspetta che MergeSort mantenga prestazioni più regolari e QuickSort possa andare incontro a peggioramenti su campi stringa.

Esercizio 2: Edit Distance

La funzione `edit_distance(const char *s1, const char *s2)` restituisce, come caso base, la lunghezza della stringa residua se una delle due è vuota. Il caso ricorsivo calcola:

- `d_noop`: se i primi caratteri coincidono, prosegue senza operazioni, altrimenti viene impostato a un valore elevato;
- `d_canc`: corrisponde a una cancellazione su `s2`;
- `d_ins`: corrisponde a un inserimento su `s1`.

La funzione restituisce il minimo tra questi tre valori.

La funzione `edit_distance_dyn` utilizza memoization tramite una matrice bidimensionale inizializzata a -1. Ogni sottoproblema viene calcolato una sola volta e memorizzato, riducendo la complessità da esponenziale a $O(|s1| * |s2|)$.

Il file `main_ex2.c` implementa un programma che riceve come parametri il dizionario e il file da correggere. Il dizionario viene caricato in memoria e il testo viene suddiviso parola per parola. Per ogni parola del testo, si calcola la distanza di edit rispetto a tutte quelle del dizionario usando `edit_distance_dyn`. Vengono proposte come suggerimenti le parole con distanza minima.

Gli unit test sono stati implementati nel file `edit_distance_test.c` utilizzando il framework Unity. Sono stati verificati diversi casi:

- stringhe vuote (distanza 0);
- una stringa vuota e una non vuota;
- stringhe identiche (distanza 0);
- operazioni di inserimento, cancellazione e combinazioni;
- confronto tra implementazione ricorsiva e dinamica, con risultati sempre coincidenti.

Per ciascuna parola non presente nel dizionario, il programma ha restituito una lista di possibili correzioni con distanza minima. Alcuni esempi:

- parola 'vinoo' → suggerita 'vino' (distanza 1);
- parola 'cassa' → suggerita 'casa' (distanza 1);
- parola 'passatto' → suggerita 'passato' (distanza 1).

La versione dinamica si è dimostrata molto più efficiente, specialmente su testi e dizionari di dimensioni maggiori.

Esercizio 3: PriorityQueue

Implementazione di una coda che supporta le operazioni di inserimento, rimozione, accesso all'elemento con priorità minima/massima e verifica di appartenenza, rispettando le complessità richieste dall'interfaccia *AbstractQueue<E>*.

La struttura principale è un heap binario realizzato tramite *ArrayList<E>*, che garantisce:

push e pop in $O(\log N)$ e top in $O(1)$

Per supportare *contains(e)* e *remove(e)* in $O(1)$ si è affiancato una *HashMap<E,Integer>* che associa ogni elemento alla sua posizione nello heap.

In questo modo:

- *contains* diventa una semplice lookup nella mappa.
- *remove(e)* sfrutta la posizione nota per rimuovere l'elemento in $O(\log N)$ (scambio con ultimo elemento e ripristino heap).

L'uso congiunto di heap + mappa di indici è una implementazione per la priority queue.

Operazione	Complessità
empty	$O(1)$
push	$O(\log N)$
contains	$O(1)$
top	$O(1)$
pop	$O(\log N)$
remove(e)	$O(\log N)$

Abbiamo implementato test con JUnit:

- Inserimento multiplo e verifica dell'ordine.
- Rimozione del minimo e controllo che il nuovo minimo sia corretto.
- Test di *contains* su elementi presenti e non presenti.
- Rimozione arbitraria e verifica della struttura.
- Test con oggetti personalizzati e Comparator personalizzati.

Esercizio 4: Grafi sparsi e foreste ricoprenti minime

Realizzazione di una libreria per grafi sparsi con gestione di archi diretti/non diretti e archi etichettati/non etichettati.

Successivamente usiamo questa libreria insieme alla *PriorityQueue* dell'esercizio 3 per implementare l'algoritmo di **Prim** e calcolare una foresta ricoprente minima.

Struttura dati grafo: Abbiamo utilizzato una lista di adiacenza rappresentata con: $Map<V, Map<V, L>> adj;$

- $V \rightarrow$ nodo
- $Map<V, L> \rightarrow$ nodi adiacenti con relativa etichetta (peso arco)

Gestione diretto/non diretto

- Se il grafo è non diretto, ogni arco (a, b) viene duplicato come (b, a).
- Se il grafo è non etichettato, le etichette vengono mantenute come null.

Algoritmo di **Prim**

- L'algoritmo è stato implementato generico su $<V, L \text{ extends } Number>$.
- Usa la *PriorityQueue* dell'esercizio 3 per selezionare l'arco minimo uscente.
- Gestisce il caso di più componenti connesse, che produce una foresta.
- Complessità: $O(M \log N)$, ottimale per grafi sparsi.

Operazione	Complessità
addNode	$O(1)$
addEdge	$O(1)$ (supponendo lista adiacenze corta)
containsNode	$O(1)$
containsEdge	$O(1)$
removeNode	$O(N)$
removeEdge	$O(1)$
numNodes	$O(1)$
numEdges	$O(N)$
getNeighbours	$O(1)$
getLabel	$O(1)$

Uso su dataset reale

Abbiamo testato l'algoritmo di Prim sul file *italian_dist_graph.csv*:

- Nodi: ~18.640
- Archi MST: 18.637
- Peso totale ≈ 89.939 km

Test

- Verifica creazione e gestione nodi/archi.
- Test con grafi diretti vs non diretti.
- Test su grafi etichettati vs non etichettati.
- Verifica del numero di nodi e archi nella foresta risultante.